



UNIVERSIDAD DE LAS FUERZAS ARMADAS

ESPE

Departamento de Ciencias de la Computación.

INTEGRANTES: Martin Palacios
Matias Medina
Leopoldo Bolanos

FECHA: 18 de junio del 2026.

GRUPO: Grupo 7

ASIGNATURA: Metodología de desarrollo de Software.

Modelado UML, Diagrama de casos de uso, Diagrama Puente

DOCENTE: Mgtr. Jenny Ruiz.

NRC: 30693

1.	REQUISITOS FUNCIONALES POR SPRINT	4
2.	SPRINT 1	4
2.1.	DIAGRAMA DE CASOS DE USO	4
2.2.	DIAGRAMA PUENTE.....	6
2.3.	DIAGRAMA DE CLASES	8
3.	SPRINT 2	11
3.1.	DIAGRAMA CASOS DE USO.....	11
3.2.	DIAGRAMA PUENTE.....	12
3.3.	DIAGRAMA DE CLASES	14
4.	SPRINT 3	18
4.1.	DIAGRAMA DE CASOS DE USO	18
4.2.	DIAGRAMA PUENTE.....	19
4.3.	DIAGRAMA DE CLASES	20

1. Listado de Requisitos del Sistema de Gestión de Caja para TAGS y Productos

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
REQ001	El sistema deberá permitir el acceso del administrador mediante usuario y contraseña.	Administrador/ Cajero	Iniciar sesión
REQ002	El sistema deberá permitir la gestión de productos de tienda para agregarlos al cierre de caja.	Administrador/ Cajero	Gestionar productos
REQ003	El sistema deberá permitir el registro de clientes con validación de datos.	Administrador/ Cajero	Registrar cliente
REQ004	El sistema deberá permitir la gestión de servicios TAG.	Administrador/ Cajero	Gestionar servicios TAG
REQ005	El sistema deberá permitir la gestión de servicios variables con concepto y valor.	Administrador/ Cajero	Gestionar servicios variables
REQ006	El sistema deberá permitir consultar y guardar el cierre final de caja en una base de datos no relacional.	Administrador/ Cajero	Cierre de caja
REQ007	El sistema debe permitir ingresar un valor numérico llamado caja chica en el modulo de cierre de caja	Administrador/ Cajero	Caja Chica
REQ008	El sistema deberá permitir buscar por fecha los cierres de caja existentes.	Administrador/ Cajero	Busqueda por Fecha
REQ009	El sistema deberá permitir filtrar por método de pago las ventas que se realice durante una jornada para poder diferenciar el dinero que se tiene en efectivo	Administrador/ Cajero	Metodo de Pago

1. Requisitos Funcionales por Sprint

Sprint	Requisito funcional
Sprint 1	REQ001 - Acceso al sistema
Sprint 1	REQ002 - Gestión de productos
Sprint 1	REQ003 - Registro de clientes
Sprint 2	REQ004 - Gestión de servicios TAG
Sprint 2	REQ005 - Gestión de servicios variables
Sprint 2	REQ006 - Cierre de caja
Sprint 3	REQ007 – Gestion Caja Chica
Sprint 3	REQ008 - Búsqueda de Cierres de Caja por fecha
Sprint 3	REQ009 – Gestion de Metodos de Pago

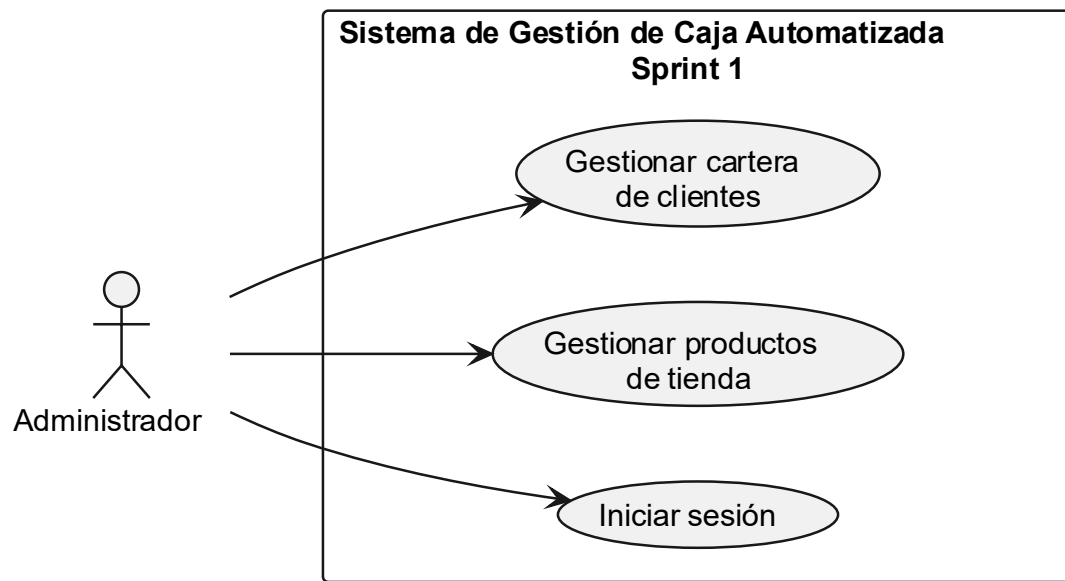
2. Sprint 1

2.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml CasosUso_Sprint1
left to right direction
actor "Administrador" as Admin
rectangle "Sistema de Gestión de Caja Automatizada\nSprint 1" {
    usecase "Iniciar sesión" as UC1
    usecase "Gestionar productos\nde tienda" as UC2
    usecase "Gestionar cartera\nde clientes" as UC3
}
Admin --> UC1
Admin --> UC2
Admin --> UC3

@enduml
```

Diagrama de casos de uso en PlantUML

2.2.Diagrama Puente

Código Puente PlantUML

```

@startuml Puente_Sprint1
left to right direction
actor "Administrador" as Admin
boundary "LoginView" as LoginView
control "LoginController" as LoginController
entity "UserModel" as UserModel

boundary "MainView" as MainView
boundary "SalesPanel" as SalesPanel
control "SalesController" as SalesController
entity "CurrentSaleModel" as CurrentSaleModel
boundary "ClientPanel" as ClientPanel

control "ClientController" as ClientController
entity "ClientModel" as ClientModel
control "ClientRepository" as ClientRepository
database "MongoDB" as MongoDB

Admin --> LoginView : ingresa credenciales
LoginView --> LoginController : envía usuario y contraseña

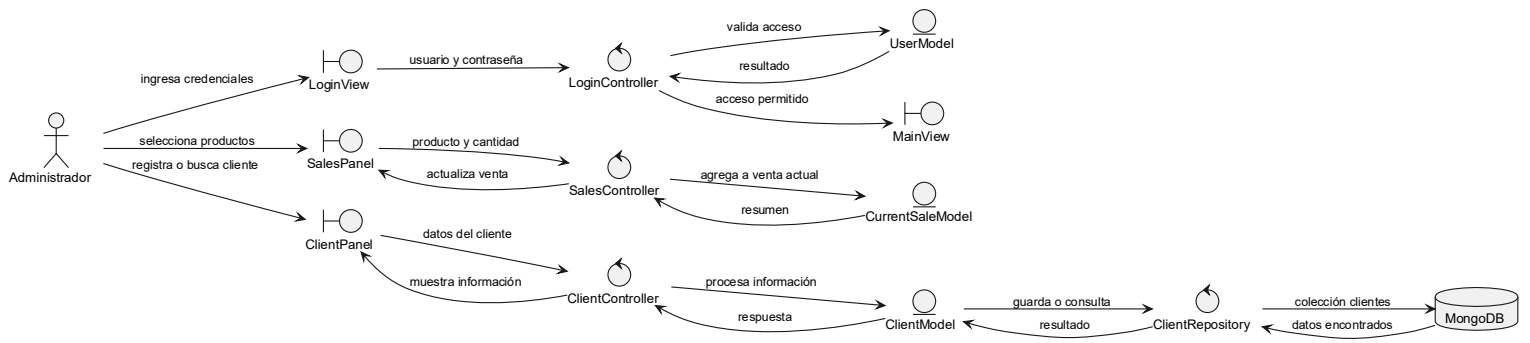
LoginController --> UserModel : valida acceso
UserModel --> LoginController : retorna resultado
LoginController --> MainView : abre menú principal
Admin --> SalesPanel : selecciona productos
SalesPanel --> SalesController : envía producto y cantidad
SalesController --> CurrentSaleModel : agrega a venta actual
CurrentSaleModel --> SalesController : retorna resumen
SalesController --> SalesPanel : actualiza pantalla

Admin --> ClientPanel : registra o busca cliente
ClientPanel --> ClientController : envía datos del cliente
ClientController --> ClientModel : valida y procesa
ClientModel --> ClientRepository : guarda o consulta
ClientRepository --> MongoDB : opera en colección clientes
MongoDB --> ClientRepository : retorna datos
ClientRepository --> ClientModel : entrega resultado
ClientModel --> ClientController : retorna respuesta
ClientController --> ClientPanel : muestra resultado

@enduml

```

Diagrama de Puente



2.3.Diagrama de clases

Código en PlantUML

```
@startuml Clases_Sprint1_ProgramaCaja

skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
left to right direction

title Diagrama de Clases - Sprint 1\nAcceso, Productos y Cartera de Clientes

package "view" {
    class LoginView <<View>> {
        + getTxtUsuario()
        + getTxtPassword()
        + getBtnLogin()
    }

    class MainView <<View>> {
        + MainView()
    }

    class SalesPanel <<View>> {
        + actualizarResumen()
    }

    class ClientPanel <<View>> {
        + getTxtCedula()
        + getTxtNombres()
        + getTxtCelular()
        + getTxtCorreo()
        + mostrarClientes()
    }
}

package "controller" {
    class LoginController <<Controller>> {
        + LoginController()
    }

    class SalesController <<Controller>> {
        + agregarProducto()
        + agregarAVentaActual()
    }
}
```

```

class ClientController <<Controller>> {
    + guardarCliente()
    + buscarPorCedula()
    + mostrarTodos()
    + limpiarFormulario()
}

```

```

package "model" {
    class UserModel <<Model>> {
        + validarCredenciales()
    }
}

```

```

class CurrentSaleModel <<Model>> {
    + addItem()
    + hasItems()
    + getTotal()
    + getReceiptText()
    + clearSale()
}

```

```

class CurrentSaleItem <<Model>> {
    + getDescription()
    + getQuantity()
    + getUnitPrice()
    + getSubtotal()
}

```

```

enum SaleItemType {
    PRODUCT
    TAG_SERVICE
    VARIABLE_SERVICE
}

```

```

class ClientModel <<Model>> {
    + saveClient()
    + findByCedula()
    + existsByCedula()
    + getCientesList()
}

```

```

package "database" {

```

```

class ClientRepository <<Repository>> {
    + saveClient()
    + findByCedula()
    + existsByCedula()
    + getCientesList()
}

class MongoConnection <<Database>> {
    + getDatabase()
    + closeConnection()
}
}

ProgramaCaja --> LoginView : inicia
LoginView --> LoginController : eventos login
LoginController --> UserModel : valida usuario
LoginController --> MainView : acceso correcto

MainView --> SalesPanel : muestra módulo
SalesPanel --> SalesController : eventos productos
SalesController --> CurrentSaleModel : agrega productos
CurrentSaleModel --> CurrentSaleItem : contiene
CurrentSaleItem --> SaleItemType : clasifica

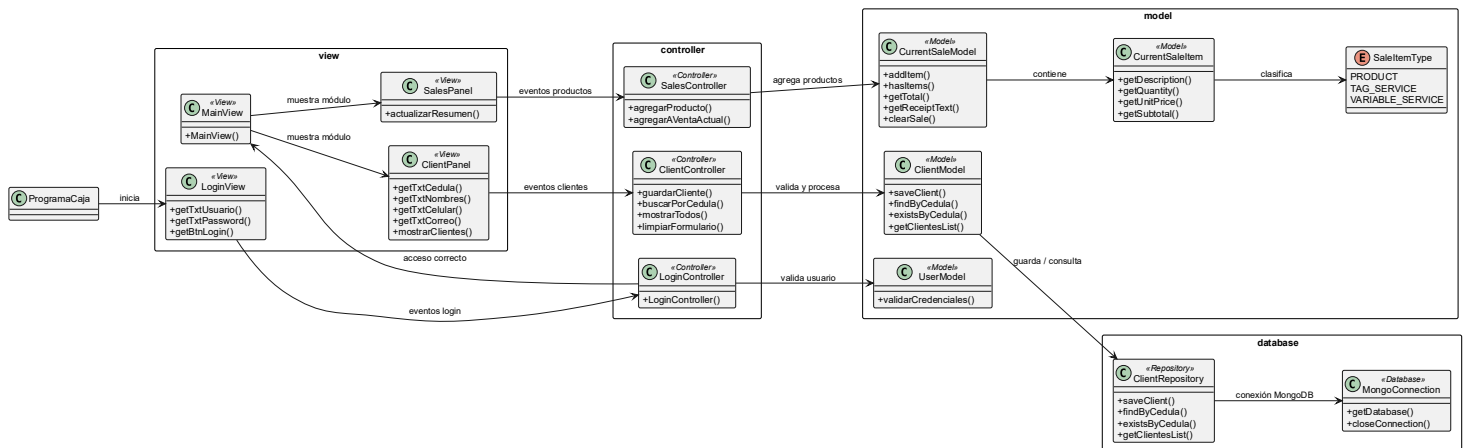
MainView --> ClientPanel : muestra módulo
ClientPanel --> ClientController : eventos clientes
ClientController --> ClientModel : valida y procesa
ClientModel --> ClientRepository : guarda / consulta
ClientRepository --> MongoConnection : conexión MongoDB

@enduml

```

Diagrama de clases

Diagrama de Clases - Sprint 1
Acceso, Productos y Cartera de Clientes



3. Sprint 2

3.1. Diagrama Casos de Uso

Codigo en PlantUML

```
@startuml CasosUso_Sprint2
```

left to right direction

actor "Administrador" as Admin

```

rectangle "Sistema de Gestión de Caja Automatizada\nSprint 2" {
    usecase "Gestionar servicios\nTAG" as UC4
    usecase "Gestionar servicios\nvariables" as UC5
    usecase "Generar cierre\nde caja" as UC6
}

```

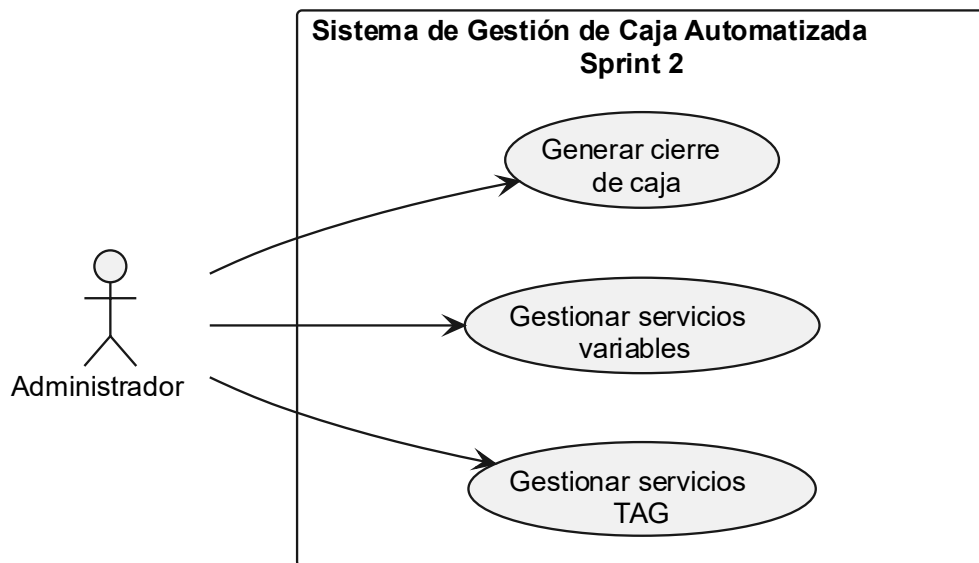
Admin --> UC4

Admin --> UC5

Admin --> UC6

```
@enduml
```

Diagrama de casos de uso en PlantUML



3.2.Diagrama Puente

Codigo Puente PlantUML

```
@startuml Puente_Sprint2
```

left to right direction

```
actor "Administrador" as Admin
```

```
boundary "TagServicePanel" as TagServicePanel
```

```
control "TagServiceController" as TagServiceController
```

```
boundary "VariableServicePanel" as VariableServicePanel
```

```
control "VariableServiceController" as VariableServiceController
```

```
entity "CurrentSaleModel" as CurrentSaleModel
```

```
boundary "CashFlowPanel" as CashFlowPanel
```

```
control "CashFlowController" as CashFlowController
```

```
entity "CashFlowModel" as CashFlowModel
```

```
control "CashFlowRepository" as CashFlowRepository
```

```
database "MongoDB" as MongoDB
```

Admin --> TagServicePanel : selecciona servicio TAG

TagServicePanel --> TagServiceController : servicio y cantidad

TagServiceController --> CurrentSaleModel : agrega a venta actual

CurrentSaleModel --> TagServiceController : resumen

TagServiceController --> TagServicePanel : actualiza pantalla

Admin --> VariableServicePanel : ingresa servicio variable

VariableServicePanel --> VariableServiceController : concepto y valor

VariableServiceController --> CurrentSaleModel : agrega a venta actual

CurrentSaleModel --> VariableServiceController : resumen

VariableServiceController --> VariableServicePanel : actualiza pantalla

Admin --> CashFlowPanel : consulta o guarda cierre

CashFlowPanel --> CashFlowController : solicita reporte

CashFlowController --> CashFlowModel : obtiene cierre de caja

CashFlowModel --> CashFlowController : reporte generado

CashFlowController --> CashFlowPanel : muestra reporte

CashFlowController --> CashFlowModel : guardar jornada

CashFlowModel --> CashFlowRepository : envía cierre

CashFlowRepository --> MongoDB : guarda en cierres_caja

MongoDB --> CashFlowRepository : confirmación

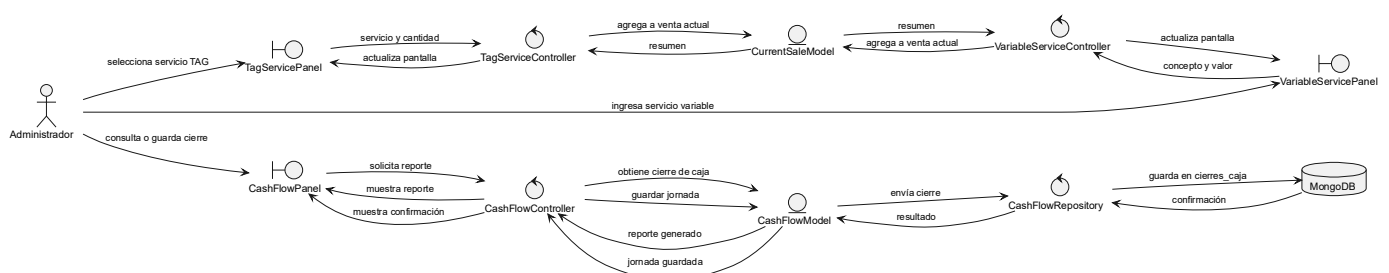
CashFlowRepository --> CashFlowModel : resultado

CashFlowModel --> CashFlowController : jornada guardada

CashFlowController --> CashFlowPanel : muestra confirmación

@enduml

Diagrama de puente en PlantUML



3.3.Diagrama de clases

Codigo Diagrama de clases PlantUML4

```
@startuml Clases_Sprint2_ProgramaCaja

skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
left to right direction

title Diagrama de Clases - Sprint 2\nServicios TAG, Servicios Variables y Cierre de Caja

package "view" {
    class MainView <<View>> {
        + MainView()
    }

    class TagServicePanel <<View>> {
        + actualizarResumen()
    }

    class VariableServicePanel <<View>> {
        + getConcepto()
        + getValor()
        + actualizarResumen()
    }

    class CashFlowPanel <<View>> {
        + actualizarReporte()
        + mostrarReporte()
    }
}

package "controller" {
    class TagServiceController <<Controller>> {
        + agregarServicioTAG()
        + agregarAVentaActual()
    }
}
```

```

    + limpiarServicios()
}

class VariableServiceController <<Controller>> {
    + agregarServicioVariable()
    + agregarAVentaActual()
    + limpiarServicios()
}

class CashFlowController <<Controller>> {
    + actualizarReporte()
    + guardarJornada()
}

package "model" {
    class CurrentSaleModel <<Model>> {
        + addItem()
        + hasItems()
        + getTotal()
        + getReceiptText()
        + clearSale()
    }

    class CurrentSaleItem <<Model>> {
        + getDescription()
        + getQuantity()
        + getUnitPrice()
        + getSubtotal()
    }

    enum SaleItemType {
        PRODUCT
        TAG_SERVICE
        VARIABLE_SERVICE
    }

    class CashFlowModel <<Model>> {
        + hasMovements()
    }
}

```

```

    + getFinalReport()
    + getGrandTotal()
    + saveJournalAndReset()
}
}

```

```

package "database" {

    class CashFlowRepository <<Repository>> {
        + saveCashClosure()
        + getSavedJournalsList()
    }

    class MongoConnection <<Database>> {
        + getDatabase()
        + closeConnection()
    }
}

```

MainView --> TagServicePanel : muestra módulo

TagServicePanel --> TagServiceController : eventos TAG

TagServiceController --> CurrentSaleModel : agrega servicio TAG

MainView --> VariableServicePanel : muestra módulo

VariableServicePanel --> VariableServiceController : eventos variables

VariableServiceController --> CurrentSaleModel : agrega servicio variable

CurrentSaleModel --> CurrentSaleItem : contiene

CurrentSaleItem --> SaleItemType : clasifica

MainView --> CashFlowPanel : muestra módulo

CashFlowPanel --> CashFlowController : eventos cierre

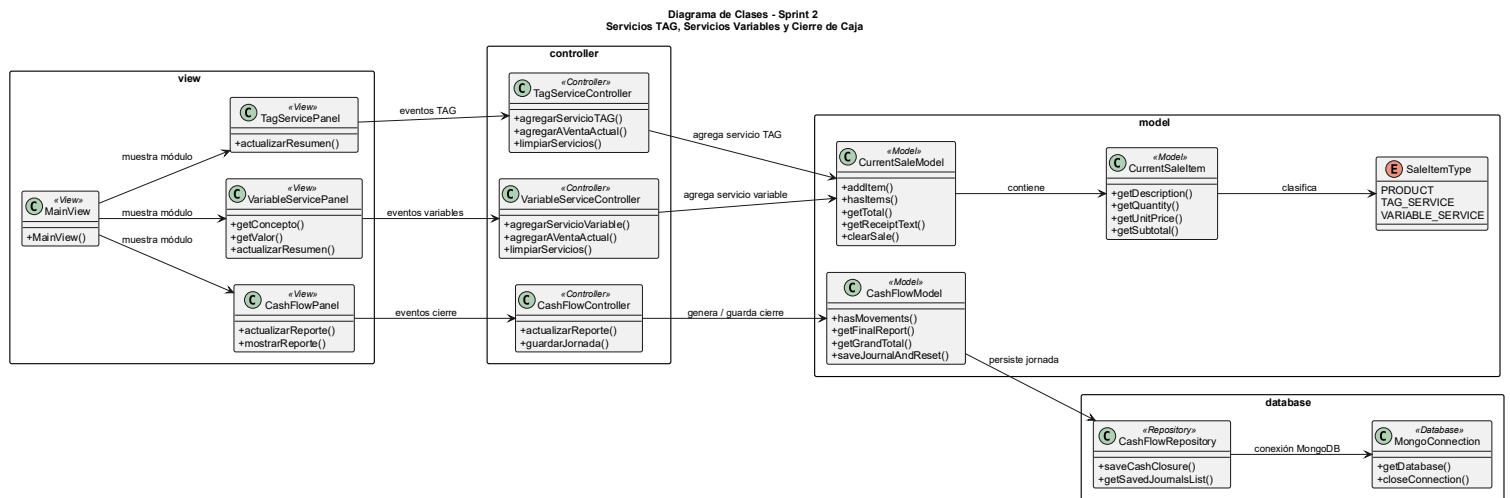
CashFlowController --> CashFlowModel : genera / guarda cierre

CashFlowModel --> CashFlowRepository : persiste jornada

CashFlowRepository --> MongoConnection : conexión MongoDB

@enduml

Diagrama de clases en PlantUML



4. Sprint 3

4.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml CasosUso_Sprint3

left to right direction

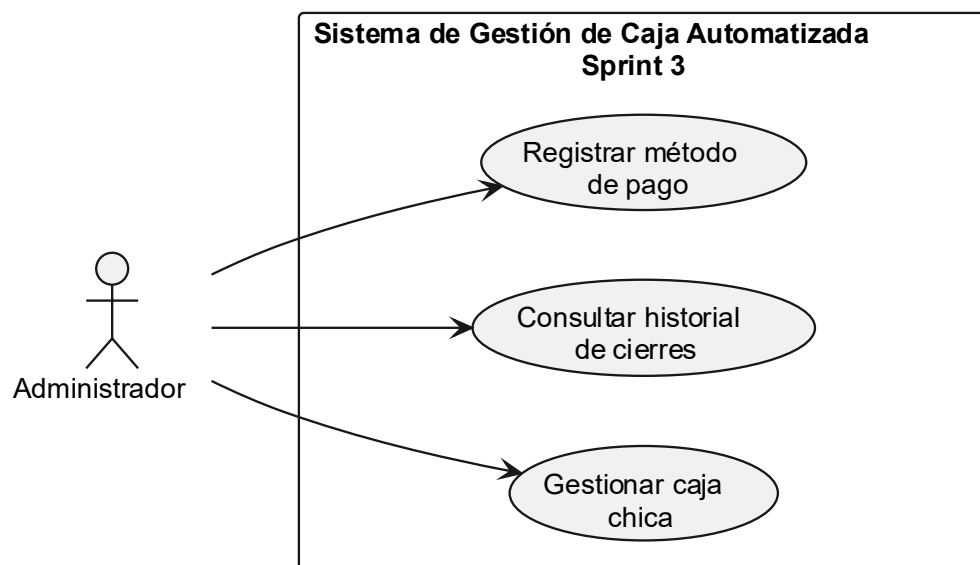
actor "Administrador" as Admin

rectangle "Sistema de Gestión de Caja Automatizada\nSprint 3" {
    usecase "Gestionar caja\nchica" as UC7
    usecase "Consultar historial\nde cierres" as UC8
    usecase "Registrar método\nde pago" as UC9
}

Admin --> UC7
Admin --> UC8
Admin --> UC9

@enduml
```

Diagrama de casos de uso en PlantUML



4.2.Diagrama Puente

Código Puente PlantUML

```
@startuml Puente_Sprint3_Vertical
```

```
top to bottom direction
```

```
actor "Administrador" as Admin
```

```
boundary "PaymentPanel" as PaymentPanel
```

```
control "PaymentController" as PaymentController
```

```
entity "CurrentSaleModel" as CurrentSaleModel
```

```
entity "PaymentDetail" as PaymentDetail
```

```
boundary "CashFlowPanel" as CashFlowPanel
```

```
control "CashFlowController" as CashFlowController
```

```
entity "CashFlowModel" as CashFlowModel
```

```
boundary "CashHistoryPanel" as CashHistoryPanel
```

```
control "CashHistoryController" as CashHistoryController
```

```
entity "CashHistoryModel" as CashHistoryModel
```

```
control "CashFlowRepository" as CashFlowRepository
```

```
database "MongoDB" as MongoDB
```

```
Admin --> PaymentPanel : selecciona método de pago
```

```
PaymentPanel --> PaymentController : venta y forma de pago
```

```
PaymentController --> CurrentSaleModel : consulta venta actual
```

```
PaymentController --> PaymentDetail : genera detalle de pago
```

```
PaymentController --> CashFlowModel : envía venta pagada
```

```
CashFlowModel --> PaymentController : confirma registro
```

```
PaymentController --> PaymentPanel : muestra confirmación
```

```
Admin --> CashFlowPanel : registra caja chica
```

```
CashFlowPanel --> CashFlowController : valores de caja chica
```

```
CashFlowController --> CashFlowModel : actualiza caja chica
```

```
CashFlowModel --> CashFlowRepository : consulta cierre anterior
```

```
CashFlowRepository --> MongoDB : busca caja chica previa
```

```
MongoDB --> CashFlowRepository : retorna datos
```

```
CashFlowRepository --> CashFlowModel : caja chica anterior
```

```
CashFlowModel --> CashFlowController : resultado
```

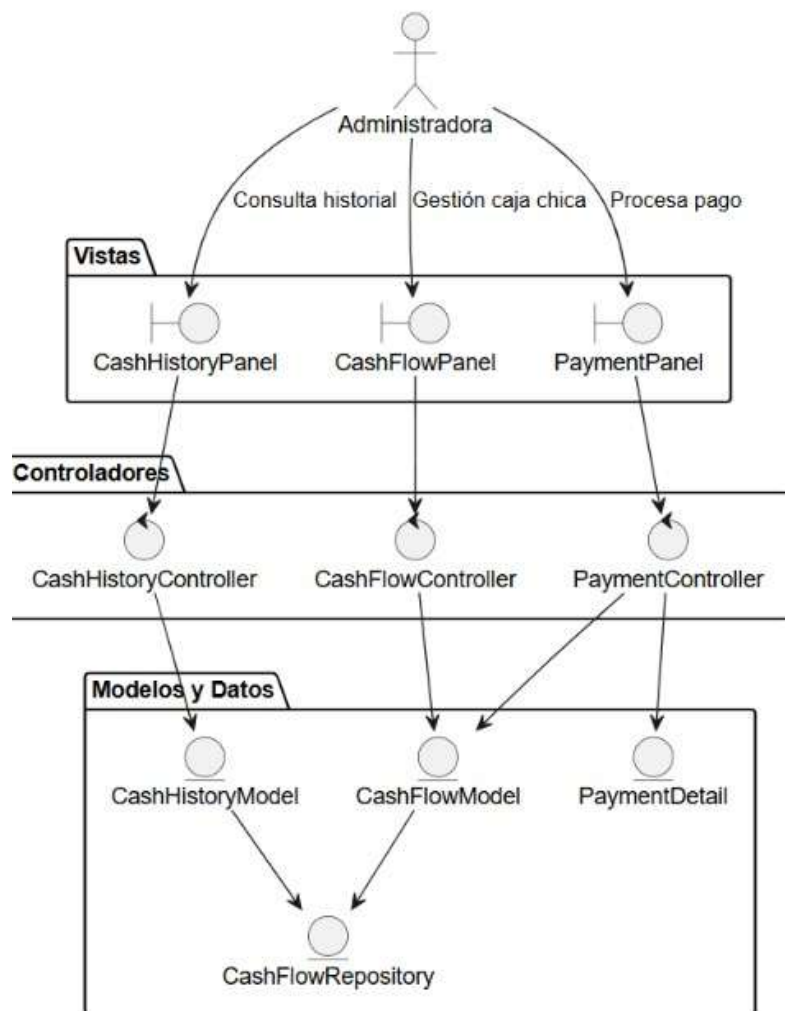
```
CashFlowController --> CashFlowPanel : actualiza pantalla
```

```
Admin --> CashHistoryPanel : busca cierre por fecha
```

CashHistoryPanel --> CashHistoryController : fecha ingresada
 CashHistoryController --> CashHistoryModel : solicita historial
 CashHistoryModel --> CashFlowRepository : consulta cierres
 CashFlowRepository --> MongoDB : busca en cierres_caja
 MongoDB --> CashFlowRepository : retorna cierres
 CashFlowRepository --> CashHistoryModel : resultados
 CashHistoryModel --> CashHistoryController : historial encontrado
 CashHistoryController --> CashHistoryPanel : muestra reporte

@enduml

Diagrama puente en PlantUML



4.3. Diagrama de clases

Código diagrama de clases PlantUML

```
@startuml Clases_Sprint3_ProgramaCaja
```

```
skinparam classAttributeIconSize 0
```

```
skinparam packageStyle rectangle
```

```
left to right direction
```

```
title Diagrama de Clases - Sprint 3\nCaja Chica, Historial y Métodos de Pago
```

```
package "view" {
```

```
    class MainView <<View>> {
```

```
        + MainView()
```

```
    }
```

```
    class PaymentPanel <<View>> {
```

```
        + getSelectedPaymentMethod()
```

```
        + getCashAmountText()
```

```
        + getTransferAmountText()
```

```
        + getCardAmountText()
```

```
        + setSaleSummary()
```

```
        + clearPaymentFields()
```

```
    }
```

```
    class CashFlowPanel <<View>> {
```

```
        + getCajaChicaActual()
```

```
        + getCajaChicaManana()
```

```
        + actualizarReporteAutomatico()
```

```
        + mostrarReporte()
```

```
    }
```

```
    class CashHistoryPanel <<View>> {
```

```
        + getFechaBusqueda()
```

```
        + mostrarResultados()
```

```
    }
```

```
}
```

```
package "controller" {
```

```
    class PaymentController <<Controller>> {
```

```
        + refreshSaleSummary()
```

```
        + confirmPayment()
```

```
        + clearCurrentSale()
```

```
    }
```

```
    class CashFlowController <<Controller>> {
```

```
        + guardarCajaChica()
```

```

        + actualizarReporte()
        + guardarJornada()
    }

    class CashHistoryController <<Controller>> {
        + buscarPorFecha()
        + mostrarTodos()
    }
}

package "model" {
    class CurrentSaleModel <<Model>> {
        + hasItems()
        + getTotal()
        + getReceiptText()
        + clearSale()
    }

    class PaymentDetail <<Model>> {
        + createSinglePayment()
        + createMixedPayment()
        + getPaymentSummaryText()
        + getSaleTotal()
    }

    enum PaymentMethod {
        CASH
        TRANSFER
        CARD
        MIXED
    }

    class CashFlowModel <<Model>> {
        - currentSaleNumber
        - paidSaleReports
        + getCurrentSaleNumber()
        + addPaidSale()
        + setInitialCurrentPettyCashAmount()
        + setNextDayPettyCashAmount()
        + getFinalReport()
        + saveJournalAndReset()
    }

    class CashHistoryModel <<Model>> {

```

```

        + getSavedJournalsByDate()
        + getSavedJournalsList()
    }
}

package "database" {
    class CashFlowRepository <<Repository>> {
        + saveCashClosure()
        + getLastNextDayPettyCash()
        + getSavedJournalsByDate()
        + getSavedJournalsList()
    }

    class MongoConnection <<Database>> {
        + getDatabase()
        + closeConnection()
    }
}

MainView --> PaymentPanel : muestra módulo
PaymentPanel --> PaymentController : eventos pago
PaymentController --> CurrentSaleModel : consulta venta actual
PaymentController --> PaymentDetail : crea detalle pago
PaymentDetail --> PaymentMethod : usa método
PaymentController --> CashFlowModel : registra venta pagada

MainView --> CashFlowPanel : muestra módulo
CashFlowPanel --> CashFlowController : eventos caja chica
CashFlowController --> CashFlowModel : actualiza caja chica

MainView --> CashHistoryPanel : muestra módulo
CashHistoryPanel --> CashHistoryController : eventos historial
CashHistoryController --> CashHistoryModel : consulta cierres

CashHistoryModel --> CashFlowRepository : obtiene historial
CashFlowModel --> CashFlowRepository : guarda / consulta cierre
CashFlowRepository --> MongoConnection : conexión MongoDB

@enduml

```

Diagrama clases en PlantUML

